EduTalentX

Complete Role-Based Platform Specification

Education, Skills, Intelligence & Hiring

Purpose: Define the user roles, responsibilities, workflows, shared modules, access controls, and implementation expectations for a connected EduTalentX platform. This document is a product and implementation specification; it does not claim that these features are already deployed.

1. Platform Overview

EduTalentX is intended to connect students, institutions, faculty, recruiters, and platform operators in one role-based system. Each role has a dedicated workspace, while shared records and workflows connect the users. The system should use real persisted data, enforce permissions on the server, and clearly distinguish unavailable data from genuinely empty results.

2. User Roles and Responsibilities

2.1 Student / Developer

- Manage personal profile, education, skills, resume, projects, certificates, and achievements.
- Connect GitHub optionally and display only verifiable profile and activity data.
- Discover jobs, internships, learning resources, and placement drives; search and filter opportunities.
- Apply to opportunities and track status changes and selection stages.
- Use skill-gap analysis, career recommendations, and interview practice when configured.

2.2 Recruiter / Employer

- Maintain a company profile and manage authorized recruiter team members.
- Create, edit, publish, close, or archive job and internship listings.
- Define eligibility criteria, required skills, deadlines, locations, and selection stages.
- Review applicants, shortlist candidates, schedule interviews, and record feedback.
- Update application decisions, issue offers, and track acceptance and joining where supported.
- View hiring analytics based on authorized company data.

2.3 Teacher / Faculty

- Access assigned courses, classes, subjects, and student groups.
- Create assignments, learning activities, and skill assessments.
- Review submissions, evaluate work, and provide feedback.
- Monitor learning progress and identify potential skill gaps.
- Recommend learning resources or opportunities within assigned permissions.

2.4 Institution Admin / Placement Officer

- Manage institution details, departments, courses, batches, and student records.
- Invite students and faculty and assign institution-scoped roles.
- Coordinate placement drives and relationships with approved recruiters.
- Configure or review eligibility criteria for institution placement activities.

- Monitor application and offer progress and export authorized institution reports.

2.5 Platform Admin

- Manage user support, account review, reports, and platform moderation within granted permissions.
- Review reported profiles, listings, and inappropriate content.
- Monitor operational dashboards, errors, and service activity.
- Apply approved platform configuration changes and preserve audit records for sensitive actions.

2.6 Super Admin

- Assign and revoke platform administrator privileges.
- Manage global role-permission policies and security settings.
- Review audit logs and investigate security or abuse reports.
- Manage institution and employer verification policies.
- Access platform-wide analytics and controlled maintenance functions.

2.7 Platform Owner

- Manage subscription plans, feature entitlements, and billing configuration where implemented.
- Review platform usage, adoption, revenue, and business reports.
- Manage platform-level integrations and business configuration.
- Require explicit authorization and audit logging for high-risk settings; ownership must not imply an unlogged security bypass.

3. End-to-End Workflows

3.1 Student-to-Employment Workflow

1. Student registers, verifies email, and completes the profile.
2. Student adds education, skills, projects, certificates, and a resume; GitHub connection is optional.
3. Platform evaluates available profile evidence and may recommend learning resources or opportunities.
4. Student searches published opportunities and reviews eligibility requirements.
5. Student submits an application; the backend validates the request and prevents duplicates where applicable.
6. Recruiter reviews the application and moves it through configured selection stages.
7. Student receives appropriate status updates and sees the current stage in the application tracker.
8. An offer may be accepted or declined; joining is recorded only when confirmed by an authorized party.

3.2 Recruiter Hiring Workflow

9. Recruiter registers, verifies the account, and completes company verification if required.
10. Recruiter creates a listing with requirements, eligibility, deadline, location, and selection stages.
11. Listing is published only after any required approval or moderation.
12. Recruiter reviews eligible applications and permitted candidate evidence.
13. Recruiter shortlists candidates, schedules interviews, and records structured feedback.
14. Recruiter updates decisions and issues offers; acceptance and joining are tracked separately.

3.3 Faculty and Learning Workflow

15. Institution admin assigns faculty to relevant courses, classes, or batches.
16. Faculty creates assignments, activities, or assessments.
17. Students submit work or complete assessments.
18. Faculty evaluates submissions and provides feedback.
19. Progress records can inform learning recommendations and skill-gap analysis.

20. Authorized faculty or placement staff may refer suitable students to opportunities without exposing restricted information.

3.4 Institution Placement Workflow

21. Institution admin configures departments, courses, batches, and student records.
22. Placement officer coordinates a placement drive or an approved recruiter relationship.
23. Eligibility rules are applied to the intended student group.
24. Students apply and the placement team monitors the application pipeline.
25. Recruiters update interview and offer outcomes.
26. Institution reports aggregate authorized records within the institution's scope.

4. Application Lifecycle

Use one consistent application lifecycle across student, recruiter, and institution views. Not every opportunity must use every stage; allowed transitions should be configured and validated.

Stage	Meaning
Applied	Application submitted and persisted.
Under Review	Recruiter reviews the application and eligibility.
Shortlisted	Candidate selected for a next stage.
Interview	Interview scheduled or in progress, where applicable.
Technical Round	Technical evaluation, where applicable.
HR Round	HR discussion, where applicable.
Offer	Offer issued; acceptance is tracked separately.
Joined	Joining confirmed by an authorized recruiter.
Alternative outcomes	Rejected, withdrawn, expired, or cancelled, with appropriate transition rules and history.

5. Shared Platform Modules

Module	Required behavior
Authentication	Sign-up, sign-in, email verification, password recovery, and secure session handling.
Role-based access control	Enforce permissions on the backend; hiding a UI control is not sufficient.
Profiles	Maintain shared user identity and role-specific profile fields.
Notifications	Create relevant in-app notifications with read/unread state and appropriate recipient rules.
Email	Use the configured email provider for verification, recovery, and workflow notifications.
Search and filtering	Return only records that the current user is authorized to access.
Reporting	Calculate metrics from persisted data and scope them to authorized organizations.
Audit logs	Record sensitive administrative, hiring, and permission changes with actor and timestamp.
Integrations	Connect GitHub, AI services, payment providers, or other services only when configured and authorized.
Responsive interface	Support mobile, tablet, and desktop layouts without page-level horizontal overflow.

6. Role Connections and Data Boundaries

- Students own their profiles and applications, subject to platform rules.
- Recruiters access their company listings and applicants for those listings only.
- Faculty access assigned courses, classes, and student work only within their permissions.
- Institution admins and placement officers access records scoped to their institution.
- Platform admins, super admins, and owners receive different permissions; these roles must not be treated as interchangeable.
- Private interview notes, sensitive user information, and administrative data must not be exposed to unauthorized roles.

7. Technical Implementation Expectations

Target architecture for the existing project:

- Frontend: React + Vite with role-specific routes, reusable components, forms, validation, loading states, empty states, and error states.
- Backend: Node.js + Express APIs with authentication middleware, server-side validation, permission checks, and business logic.
- Database: MongoDB with persistent models and explicit ownership/organization relationships.
- Workflow logic: Validate eligibility, application status transitions, duplicate prevention, and actor permissions on the server.
- Notifications: Trigger appropriate in-app and email notifications after successful persisted events.
- Deployment: Use the correct production API base URL and environment variables; do not expose private secrets in frontend code.

8. Acceptance Criteria: What 'Working' Means

- A published, eligible job appears in the student opportunity list and is removed or marked unavailable when no longer active.
- When a student applies, the record appears in the authorized recruiter applicant list and the student's application tracker.
- Recruiter status changes persist after refresh and appear in the relevant authorized views.
- Institution dashboards show records only for the institution they are authorized to manage.
- If GitHub is not connected, the UI does not falsely report a connected account or fabricate repositories, commits, languages, or scores.
- If no opportunities exist, the system displays a genuine empty state and a count of zero; it does not display placeholder jobs as live records.
- API errors are distinguished from legitimate empty results.
- Unauthorized users cannot access restricted APIs by manually entering a URL or making a direct request.
- Sensitive changes are recorded in audit logs where required.
- Each workflow is tested for success, invalid input, permission denial, empty data, and service failure.

9. Recommended Implementation Phases

Phase 1 — Foundation: Authentication, user records, role permissions, organization relationships, route protection, and audit foundations.

Phase 2 — Role Workspaces: Build the seven role-specific dashboards on shared layout and API conventions.

Phase 3 — Core Workflows: Implement profiles, jobs, applications, selection stages, placement drives, and notifications end to end.

Phase 4 — Learning and Intelligence: Add assignments, assessments, skill-gap analysis, career recommendations, and GitHub integration.

Phase 5 — Operations: Add moderation, audit reports, subscriptions, billing, and integrations where required.

Phase 6 — Verification: Run end-to-end tests for every role, security checks, responsive UI checks, and deployment verification.

10. Final Recommendation

Build one shared platform with seven role-based workspaces, a common API, and shared data models. Keep permissions and business rules on the server. Implement and test each workflow end to end before moving to the next module. Verify the current repository and existing features before making changes; this document defines the intended behavior and does not assert that the functionality is already deployed.